DAMWIDI FINTECH APPLICATION
Architecture Summary and Node.js Migration Overview

Prepared: August 16, 2026

EXECUTIVE SUMMARY

Damwidi is a procedural PHP batch-and-API application for monitoring and managing a stock investment club. Its operational center is a MySQL database. A nightly cron script invokes four ordered PHP jobs that ingest club transactions, calculate security performance, reconstruct daily portfolio value, and maintain historical price data.

The application also exposes read-oriented HTTP endpoints through the same front controller used by the scheduled CLI jobs. External integrations supply club accounting data, securities prices, market calendars, logging, and alerts.

The safest migration strategy is to retain MySQL as the contract between PHP and Node.js, then replace one scheduled job at a time while comparing Node.js results with the existing PHP output.

1. RUNTIME ARCHITECTURE

The nightly execution flow is:

```
  Server cron
      |
      v
  UpdateDatabases.sh
      |
      v
  damwidiMain.php
      |
      +-- updateBivioTransactions
      +-- updatePerformanceData
      +-- updateValueTable
      +-- updateHistoryTable
             |
             +-- Bivio account scraping
             +-- Polygon market data
             +-- Tradier market calendar
             +-- MySQL reads and writes
             +-- Local JSON cache and log files
             +-- Logtail logging and IFTTT alerts
```

UpdateDatabases.sh invokes the four commands sequentially. Its use of chained success conditions means a failed command prevents the remaining commands from running.

damwidiMain.php is a shared dispatcher. When invoked from the command line, it selects a function from the first CLI argument. When invoked over HTTP, it uses the mode query parameter. php-includes/init.php eagerly loads the application modules, provider adapters, database helpers, constants, and Composer dependencies.

2. THE FOUR NIGHTLY JOBS

2.1 updateBivioTransactions

Purpose:
  Import the investment club's accounting activity.

Processing:
  - Authenticates to Bivio using cURL and a cookie file.
  - Downloads the Bivio accounting-detail CSV.
  - Reads the newest transaction date already stored in MySQL.
  - Parses and inserts only newer transactions.
  - Classifies buys, sells, dividends, payments, interest, expenses, splits,
    returns, and withdrawals.
  - Normalizes special ticker formats, such as BRKB to BRK.B.

Primary output:
  data_transactions

Architectural note:
  This stage is tightly coupled to Bivio's login form, HTML, and CSV layout.

2.2 updatePerformanceData

Purpose:
  Refresh current performance data for sectors, indexes, individual stocks,
  cash, and the Damwidi club fund.

Processing:
  - Synchronizes performance-table stock rows with currently open positions.
  - Loads two years of adjusted Polygon daily prices.
  - Calculates one- through four-day values and weekly, quarterly, annual,
    and year-to-date returns.
  - Derives basis and share counts from transaction history.
  - Builds the internally calculated DAM fund series from data_value.
  - Updates the cash record.
  - Writes a secondary JSON performance snapshot.

Primary outputs:
  data_performance
  data/performanceData.json

Architectural note:
  Provider access, financial calculations, and persistence are currently mixed
  in the same procedural workflow.

2.3 updateValueTable

Purpose:
  Reconstruct and reconcile the club's daily portfolio value.

Processing for each missing market day through yesterday:
  - Loads the exchange calendar and skips weekends and market holidays.
  - Retrieves historical SPY and held-security prices.
  - Reconstructs positions from transactions effective on that date.
  - Calculates cash and member payments.
  - Scrapes Bivio's reported unit value and units outstanding.
  - Calculates portfolio market value and per-share OHLC values.
  - Selects the available market-data result closest to Bivio's valuation.
  - Inserts or updates the daily value record.
  - Records reconciliation differences and may send a production alert.

Primary outputs:

  data_value
  Local reconciliation/discrepancy log

Architectural note:
  This is the most complex and business-critical job. It combines accounting,
  historical reconstruction, web scraping, pricing, reconciliation, storage,
  and notifications.

2.4 updateHistoryTable

Purpose:
  Incrementally maintain locally stored daily OHLC market history.

Processing:
  - Uses the first club transaction as the earliest possible history date.
  - Finds the latest stored date for each sector or index.
  - Requests only the missing interval through yesterday from Polygon.
  - Inserts the returned OHLC rows into MySQL.

Primary output:
  data_history

Architectural note:
  This is relatively isolated and is the strongest candidate for the first
  Node.js replacement.

3. PERSISTENCE MODEL

MySQL is the central integration point and system of record.

  data_transactions
    Bivio accounting ledger and the source used to reconstruct positions,
    cash, payments, and cost basis.

  data_performance
    Security metadata, current holdings, basis, share counts, recent prices,
    and calculated performance periods.

  data_value
    Daily club value, cash, payments, market value, account value, units,
    share value, Bivio value, and internally calculated OHLC data.

  data_history
    Daily OHLC records for securities, sectors, and indexes.

  data_splits
    Locally maintained stock-split records.

  data_basket
    Ancillary basket or watchlist information.

  sp_holdings and related tables
    Supporting information used by additional reporting endpoints.

There is no ORM or complete repository layer. Some SQL is centralized in
php/dataHandlerMySQL.php, while business modules also execute PDO statements
directly.

4. EXTERNAL DEPENDENCIES

Active nightly dependencies appear to be:

  Bivio
    Transactions, reported club unit values, and units outstanding.

  Polygon
    Historical and current securities prices and market status.

  Tradier
    Exchange holiday and market-calendar information.

  MySQL
    Shared operational datastore and migration boundary.

  Logtail / Better Stack
    Centralized application logging.

  IFTTT
    Production reconciliation alerts.

The repository also contains adapters or remnants for Alpha Vantage, Finnhub, IEX, BarChart, EOD Historical, AWS/Yahoo, Airtable, and MongoDB. Much of this appears to be legacy, fallback, or unrelated to the four active nightly jobs and should be verified before being reproduced in Node.js.

5. ARCHITECTURAL CHARACTERISTICS

- One monolithic bootstrap loads nearly every module for every invocation.
- Global functions and constants make dependencies implicit.
- CLI batch commands and HTTP endpoints share one front controller.
- Scraping, provider access, calculations, SQL, output, and logging are frequently mixed in individual functions.
- Jobs are incremental and generally resume from the latest database date.
- The nightly pipeline is order-dependent.
- Database writes have limited transaction boundaries, allowing partial job completion if a loop fails.
- No automated test suite is apparent in the repository.

6. RECOMMENDED GRADUAL MIGRATION

Keep the existing MySQL schema as the compatibility contract. Implement each Node.js job so it initially produces the same rows and fields as PHP. Run it in shadow mode and compare its output with the production PHP result before changing the cron schedule.

Recommended order:

1. updateHistoryTable
   Smallest scope and simplest incremental market-data ingestion.

2. updatePerformanceData
   Mostly deterministic calculations over prices and transactions.

3. updateBivioTransactions
   Logically isolated, but dependent on a fragile external scraping contract.

4. updateValueTable
   Migrate last because it contains the most important accounting and reconciliation behavior.

For the Node.js design, separate each job into explicit layers:

```
  scheduler/command -> orchestration -> domain calculations
                    -> provider adapters -> repositories -> MySQL
```

This will allow provider and calculation behavior to be tested independently while preserving the database interface used by the remaining PHP code.

7. SECURITY PRIORITY

The repository currently contains plaintext production credentials and API tokens. These should be rotated and moved to environment variables or a managed secret store before or alongside the migration. Secrets should not be copied into the Node.js implementation or committed to source control.

CONCLUSION

Damwidi is best understood as an ordered nightly data pipeline with additional read APIs attached to the same PHP entry point. MySQL provides a natural seam for gradual replacement. Starting with historical-price ingestion minimizes risk, while postponing the valuation and reconciliation engine preserves the most sensitive business logic until reliable comparison tooling exists.